Department of Computer Science & Engineering
UNIVERSITY OF LIBERAL ARTS
BANGLADESH

Lab Final Assignment

Title: Restaurant Reservation System

Course Code: CSE 2302

Course Title: Database Management System Lab

Section: 05

Submitted To:

Md. Nurul Muttakin
Lecturer
Department of Computer Science and Engineering (CSE)

Submitted By:

Name: Jarin Tasnim Eara

ID: 223014136

Date of Submission: 12 September, 2024

RESTAURANT RESERVATION SYSTEM

Introduction:

The Restaurant Reservation System database is designed to make managing restaurant operations easy and efficient. It brings together all the key elements needed for a restaurant, such as reservations, customer details, orders, menus, and administrative tasks, into a single, well-organized structure.

Imagine a busy restaurant during peak hours, where staff must juggle multiple reservations, keep track of customers, and ensure that each order is fulfilled accurately. This database acts as the backbone of the operation, making it easy for the staff to view available tables, confirm bookings, and manage customer preferences, all in real time. At the same time, customers enjoy a smooth experience, from reserving a table to placing orders, knowing their details are accurately recorded and managed.

This database uses primary keys, foreign keys, and constraints to ensure data accuracy and smooth relationships between different tables. By efficiently managing information about restaurants, customers, reservations, and orders, the system helps restaurant staff provide a better dining experience while simplifying administrative tasks.

Methodology [Database Design]:

Identifying entities and their attributes:

Primary entities:

1. Restaurant
2. TableOfRestaurant
3. Customer
4. Reservation
5. Menu
6. Order_
7. OrderDetails
8. Admin

Attributes of each entity:

1. **'Restaurant' Table :**
   - RestaurantID: An integer that uniquely identifies a restaurant. (Primary Key)
   - Name: The name of the restaurant. (Not Null)
   - Location: The location/address of the restaurant. (Not Null)

2. **'TableOfRestaurant' Table :**
   - TableID: An integer that uniquely identifies a table in the restaurant. (Primary Key)
   - RestaurantID: An integer that references the restaurant where the table is located. (Foreign Key referencing RestaurantID in the Restaurant table)
   - Capacity: The seating capacity of the table. (Not Null)

3. **'Customer' Table :**
   - CustomerID: An integer that uniquely identifies a customer. (Primary Key)
   - Name: The name of the customer. (Not Null)
   - Phone: The phone number of the customer. (Unique and Not Null)
   - Email: The email address of the customer. (Unique and Not Null)

4. **‘Reservation’ Table :**
    - ReservationID: An integer that uniquely identifies a reservation. (Primary Key)
    - CustomerID: An integer that references the customer making the reservation. (Foreign Key referencing CustomerID in the Customer table; Unique)
    - TableID: An integer that references the table reserved by the customer. (Foreign Key referencing TableID in the TableOfRestaurant table)
    - ReservationDate: The date of the reservation. (Not Null)
    - ReservationTime: The time of the reservation, restricted to a range between 09:00:00 and 22:00:00. (Not Null, Check Constraint)

5. **‘Menu’ Table :**
    - MenuItemID: An integer that uniquely identifies a menu item. (Primary Key)
    - Name: The name of the menu item. (Not Null)
    - Price: The price of the menu item. (Not Null)

6. **‘Order_’ Table :**
    - OrderID: An integer that uniquely identifies an order. (Primary Key)
    - ReservationID: An integer that references the reservation associated with the order. (Foreign Key referencing ReservationID in the Reservation table)
    - CustomerID: An integer that references the customer who made the order. (Foreign Key referencing CustomerID in the Customer table)
    - OrderDate: The date the order was made. (Not Null)

7. ‘OrderDetails’ Table :

- OrderID: An integer that references the order. (Foreign Key referencing OrderID in the Order_ table; part of Primary Key)
- MenuItemID: An integer that references the menu item ordered. (Foreign Key referencing MenuItemID in the Menu table; part of Primary Key)
- Quantity: The quantity of the menu item ordered. (Not Null)

8. ‘Admin’ Table :

- AdminID**:** An integer that uniquely identifies an admin. (Primary Key)
- Name**:** A string that holds the name of the admin. (Not Null)
- Email**:** A string that holds the admin's email address. (Unique, Not Null)
- Password**:** A string that holds the admin's password. (Not Null)
- RestaurantID**:** An integer that references the restaurant to which the admin is associated. (Foreign Key referencing RestaurantID in the Restaurant table)

ERD(Entity Relationship Diagram):

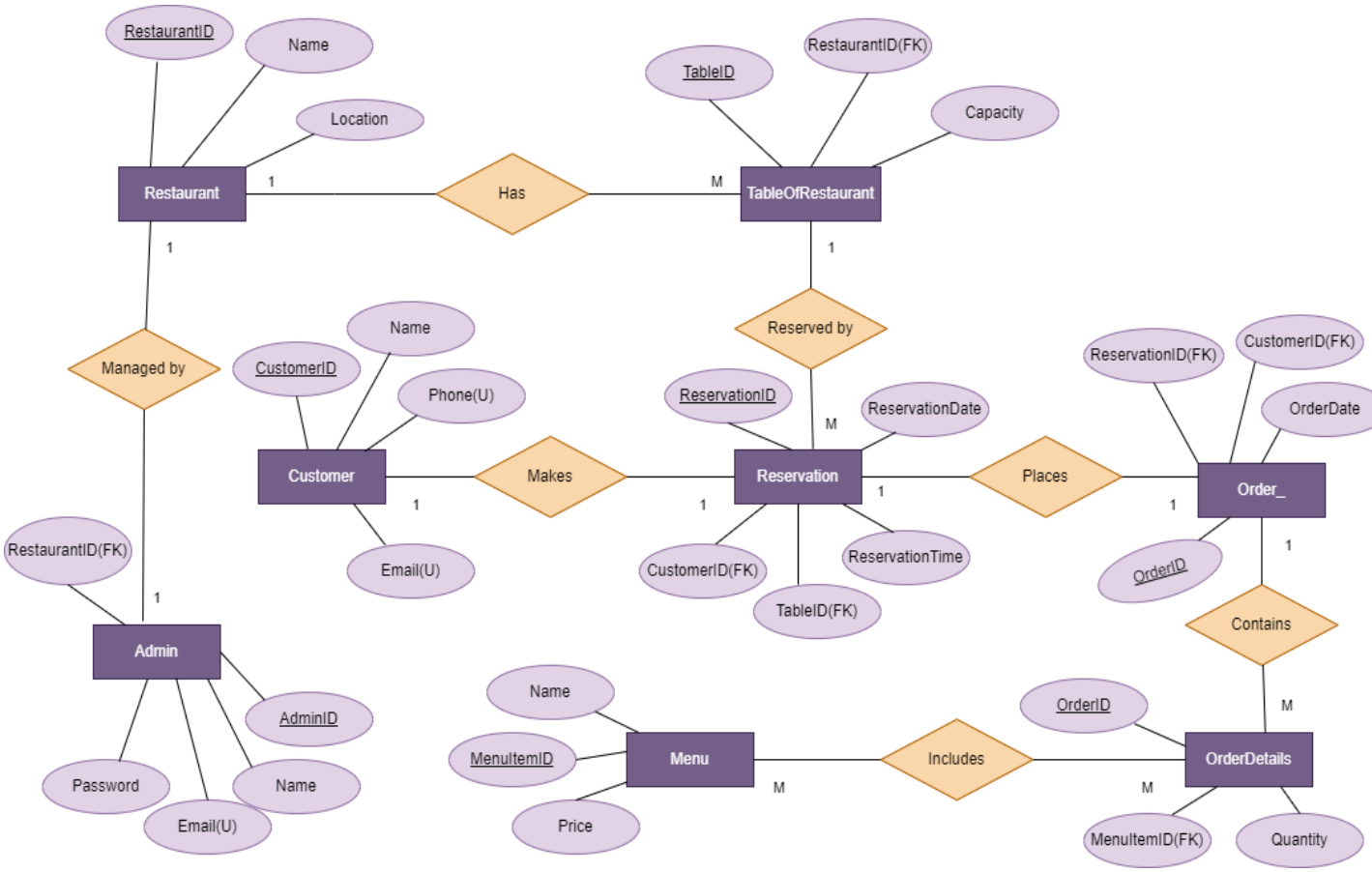


Figure : 1

In the diagram, **FK** means **Foreign Key**, **U** means **Unique** and underlined attributes are **primary Key**.

The **Restaurant Reservation System** database has no weak entities, relationships, or attributes since every entity has its own unique identifier (primary key) and doesn't rely on another entity for uniqueness or existence.

Overview of Cardinality Relationships in the Database :

One-to-Many

- Restaurant and TableOfRestaurent
- Restaurant and Admin
- TableOfRestaurent and Reservation
- Customer and Order_
- Order_ and OrderDetails

One-to-One

- Customer and Reservation
- Reservation and Order_

Many-to-Many

- Menu and OrderDetails (implemented via OrderDetails as a bridge table)

Entity Relationships and Cardinality in the System:

1. **Restaurant and TableOfRestaurent**: One restaurant can have multiple tables, but each table belongs to one restaurant. Cardinality: Restaurant(RestaurantID) → TableOfRestaurent(RestaurantID).

2. **Restaurant and Admin**: One restaurant can have multiple admins, but each admin is associated with only one restaurant. Cardinality: Restaurant(RestaurantID) → Admin(RestaurantID).

3. **Customer and Reservation**: Each customer can make one reservation at a time, linked to that customer.
   Cardinality: Customer(CustomerID) → Reservation(CustomerID).

4. **TableOfRestaurent and Reservation**: One table can have multiple reservations over time, but each reservation is for one table.
   Cardinality: TableOfRestaurent(TableID) → Reservation(TableID).

5. **Reservation and Order_**: Each reservation is linked to one order, and each order is associated with one reservation.
   Cardinality: Reservation(ReservationID) → Order_(ReservationID).

6. **Customer and Order_**: One customer can place multiple orders, but each order is linked to one customer.
   Cardinality: Customer(CustomerID) → Order_(CustomerID).

7. **Order_ and OrderDetails**: One order can have multiple items, but each detail is part of a specific order.
   Cardinality: Order_(OrderID) → OrderDetails(OrderID).

8. **Menu and OrderDetails**: Many-to-Many; menu items can appear in multiple orders, and each order can include multiple items.
   Cardinality: Menu(MenuItemID) → OrderDetails(MenuItemID).

This ER diagram shows the relationships between different entities in a restaurant reservation system. It uses primary and foreign keys to link tables, enforcing referential integrity. The diagram correctly implements cardinality, showcasing how entities interact in a one-to-one, one-to-many, or many-to-many manner. The use of unique attributes and composite keys ensures data consistency and proper database structure. This understanding of database fundamentals is crucial for designing efficient and normalized database schemas.

Implementation:

1. Data Definition Language (DDL): DDL is specification notation for defining the database schema. DDL is a set of SQL commands used to create, modify, and delete database structures but not data. These commands are normally not used by a general user, who should be accessing the database via an application.

Creating the necessary tables with constraints:

To create a database:

```
CREATE DATABASE Restaurant_Reservation_System;
```

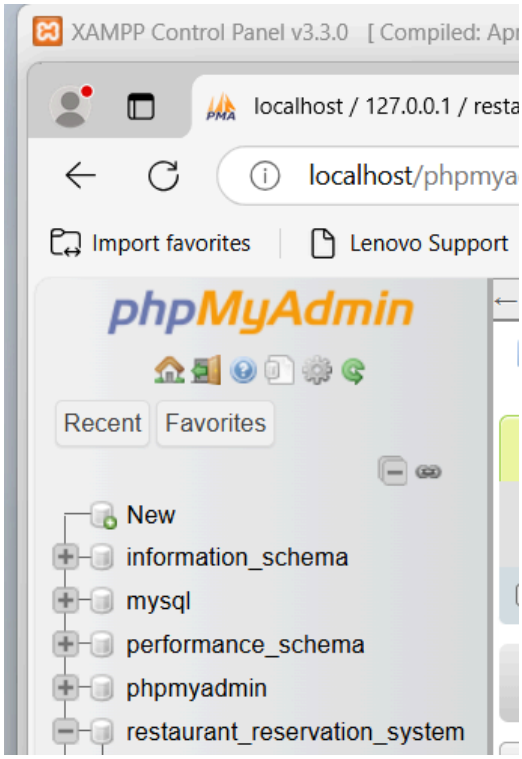


Figure 2 : created database “restaurent_reservation_system

To create Tables:

“Restaurant” table:-

```
CREATE TABLE Restaurant (
    RestaurantID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Location VARCHAR(100) NOT NULL
);
```

“TableOfRestaurent” table:-

```
CREATE TABLE TableOfRestaurent (
    TableID INT PRIMARY KEY,
    RestaurantID INT,
    Capacity INT NOT NULL,
                FOREIGN KEY (RestaurantID) REFERENCES
Restaurant(RestaurantID)
);
```

“Customer ” table:-

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15) UNIQUE NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL
);
```

“Reservation ” table:-

```
CREATE TABLE Reservation (
    ReservationID INT PRIMARY KEY,
    CustomerID INT UNIQUE,
    TableID INT,
    ReservationDate DATE NOT NULL,
    ReservationTime TIME NOT NULL,
                FOREIGN KEY (CustomerID) REFERENCES
Customer(CustomerID),
                    FOREIGN KEY (TableID) REFERENCES
tableofrestaurent(TableID),
      CONSTRAINT chk_reservation_time CHECK (ReservationTime
BETWEEN '09:00:00' AND '22:00:00')
);
```

“Menu ” table:-

```
CREATE TABLE Menu (
    MenuItemID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
```

```
    Price DECIMAL(10, 2) NOT NULL
);
```

“Order_ ” table:-

```
CREATE TABLE Order_ (
    OrderID INT PRIMARY KEY,
    ReservationID INT,
    CustomerID INT,
    OrderDate DATE NOT NULL,
                FOREIGN KEY (ReservationID) REFERENCES
Reservation(ReservationID),
                 FOREIGN KEY (CustomerID) REFERENCES
Customer(CustomerID)
);
```

“OrderDetails ” table:-

```
CREATE TABLE OrderDetails (
    OrderID INT,
    MenuItemID INT,
    Quantity INT NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES order_(OrderID),
    FOREIGN KEY (MenuItemID) REFERENCES Menu(MenuItemID),
    PRIMARY KEY (OrderID, MenuItemID)
);
```

“Admin” table:-

```
CREATE TABLE Admin (
    AdminID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Password VARCHAR(100) NOT NULL,
    RestaurantID INT,
                FOREIGN KEY (RestaurantID) REFERENCES
Restaurant(RestaurantID)
);
```

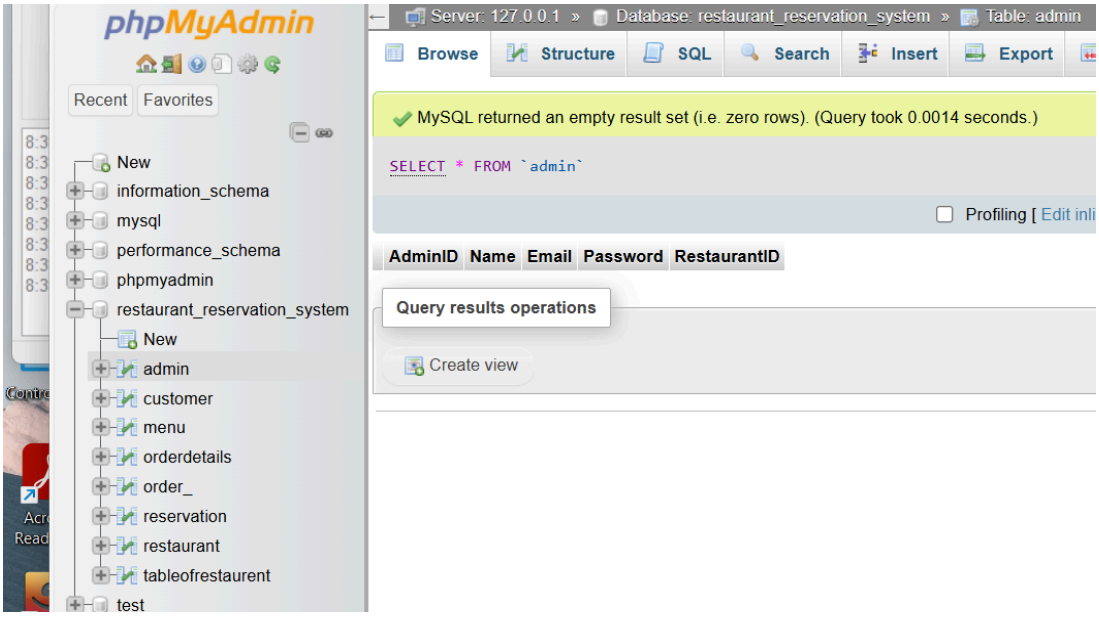


Figure 3:Created all tables of the database

2. Data Manipulation Language (DML):

Data Manipulation language is for accessing and updating the data organized by the appropriate data model.DML commands allow us to manipulate (add, update, delete, and retrieve) the data itself within the database tables.

Inserting data into the “Restaurant” table

```
INSERT INTO Restaurant (RestaurantID, Name, Location)
VALUES
    (1, 'KFC', 'Mohammadpur'),
    (2, 'Pizza Hut', 'Dhanmondi'),
    (3, 'Burger King', 'Banani'),
    (4, 'Domino\'s', 'Gulshan');
```

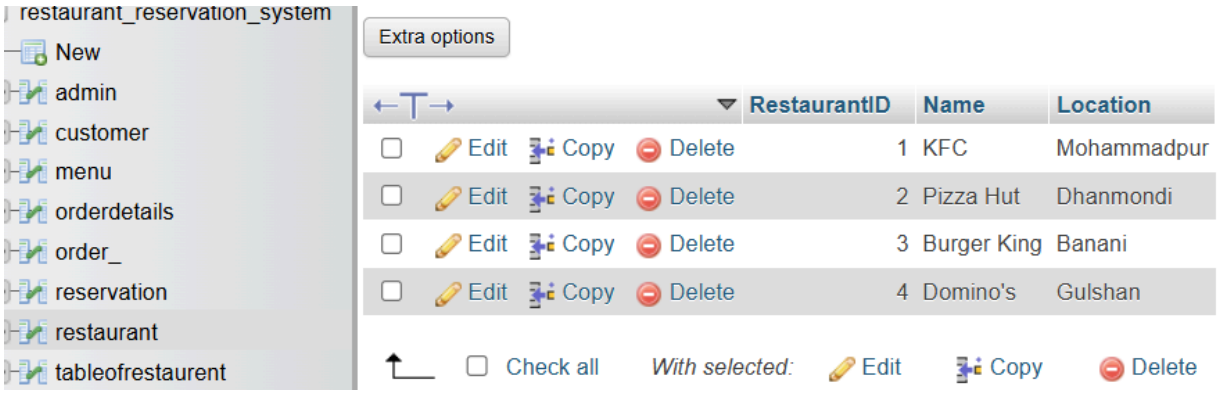


Figure :4

Inserting data into the 'TableOfRestaurent' table

```
INSERT INTO TableOfRestaurent (TableID, RestaurantID, Capacity)
VALUES
    (1, 1, 4), -- Table for 4 at KFC
    (2, 1, 2), -- Table for 2 at KFC
    (3, 2, 4), -- Table for 4 at Pizza Hut
    (4, 2, 6), -- Table for 6 at Pizza Hut
    (5, 3, 2), -- Table for 2 at Burger King
    (6, 3, 4), -- Table for 4 at Burger King
    (7, 4, 8), -- Table for 8 at Domino's
    (8, 4, 4); -- Table for 4 at Domino's
```

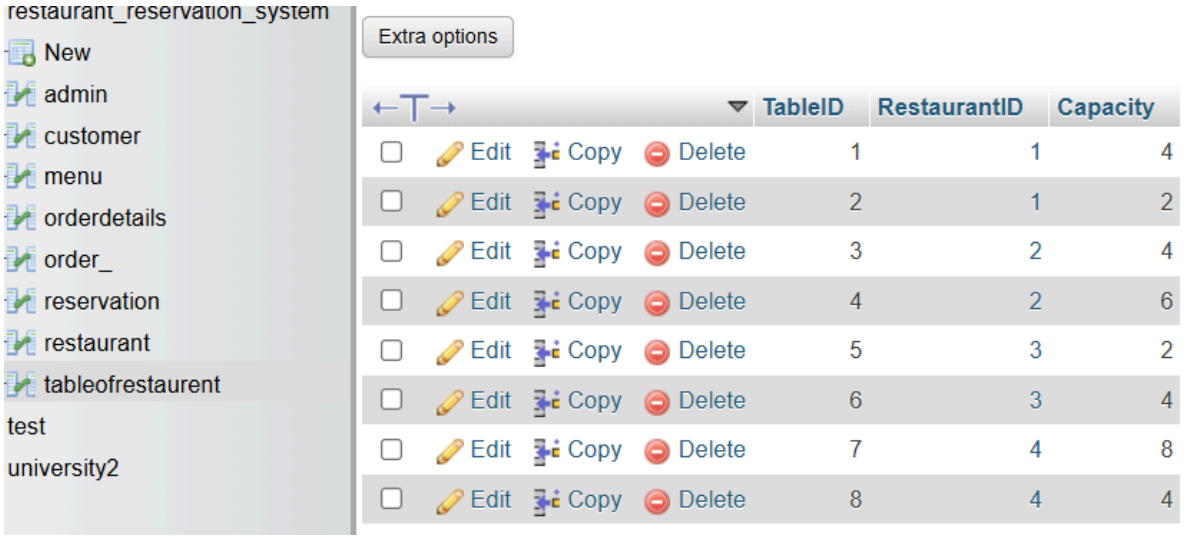
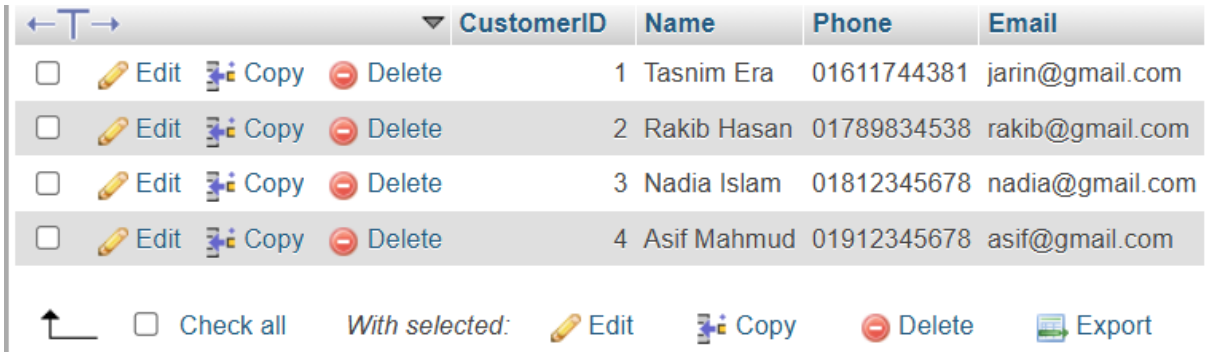


Figure :5

Inserting data into the ‘Customer’ table

```
INSERT INTO Customer (CustomerID, Name, Phone, Email)
VALUES
    (1, 'Tasnim Era', '01611744381', 'jarin@gmail.com'),
    (2, 'Rakib Hasan', '01789834538', 'rakib@gmail.com'),
    (3, 'Nadia Islam', '01812345678', 'nadia@gmail.com'),
    (4, 'Asif Mahmud', '01912345678', 'asif@gmail.com');
```



	CustomerID	Name	Phone	Email
Edit Copy Delete	1	Tasnim Era	01611744381	jarin@gmail.com
Edit Copy Delete	2	Rakib Hasan	01789834538	rakib@gmail.com
Edit Copy Delete	3	Nadia Islam	01812345678	nadia@gmail.com
Edit Copy Delete	4	Asif Mahmud	01912345678	asif@gmail.com

Check all *With selected:* Edit Copy Delete Export

Figure :6

Inserting data into the ‘Reservation’ table

```
INSERT INTO Reservation (ReservationID, CustomerID, TableID,
ReservationDate, ReservationTime)
VALUES
    (1, 1, 1, '2024-09-05', '19:00:00'), -- Tasnim Era reserving a table at
KFC
    (2, 2, 3, '2024-09-06', '18:00:00'), -- Rakib Hasan reserving a table
at Pizza Hut
    (3, 3, 5, '2024-09-07', '19:30:00'), -- Nadia Islam reserving a table at
Burger King
    (4, 4, 7, '2024-09-08', '20:00:00'); -- Asif Mahmud reserving a table
at Domino's
```

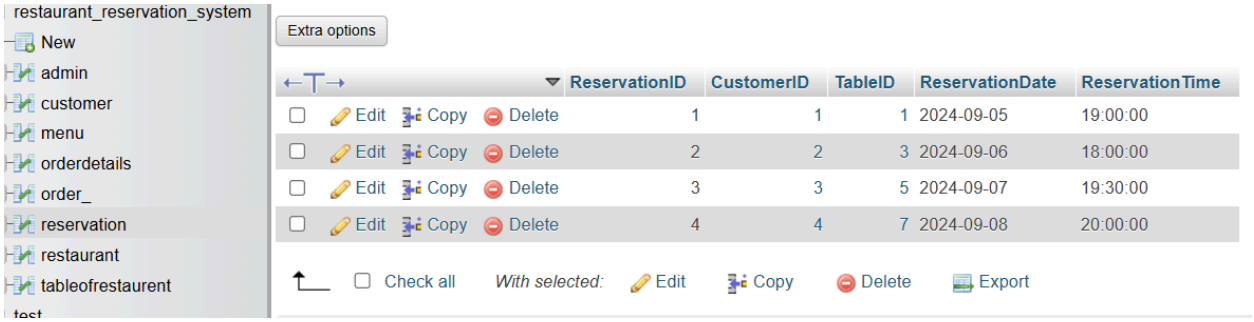


Figure :7

Inserting data into the ‘Menu’ table

```
INSERT INTO Menu (MenuItemID, Name, Price)
VALUES
    (1, 'Chicken Fry', 80.99),
    (2, 'Burger', 160.99),
    (3, 'Pizza', 250.00),
    (4, 'Pasta', 150.00),
    (5, 'Soft Drink', 50.00),
    (6, 'Ice Cream', 120.00),
    (7, 'Chicken Chowmein', 300.00),
    (8, 'Fried Rice', 80.00),
    (9, 'Caesar Salad', 180.00),
    (10, 'Wonton', 220.00),
    (11, 'Cold Coffee', 100.00),
    (12, 'Fish and Chips', 350.00),
    (13, 'French Fry', 400.00),
    (14, 'Lemonade', 60.00),
    (15, 'Thai Soup', 150.00);
```

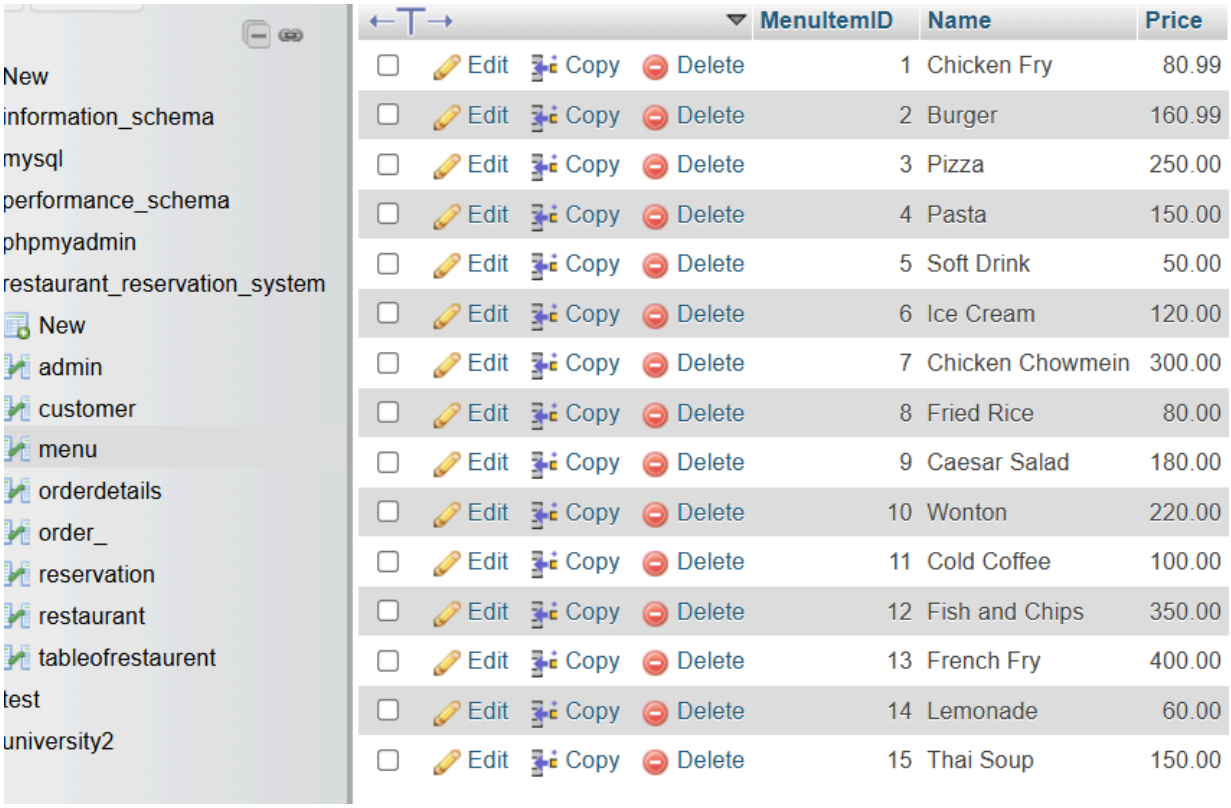


Figure :8

Inserting data into the 'Order_' table

```
INSERT INTO Order_ (OrderID, ReservationID, CustomerID, OrderDate)
VALUES
    (1, 1, 1, '2024-09-05'), -- Order for Tasnim Era
    (2, 2, 2, '2024-09-06'), -- Order for Rakib Hasan
    (3, 3, 3, '2024-09-07'), -- Order for Nadia Islam
    (4, 4, 4, '2024-09-08'); -- Order for Asif Mahmud
```

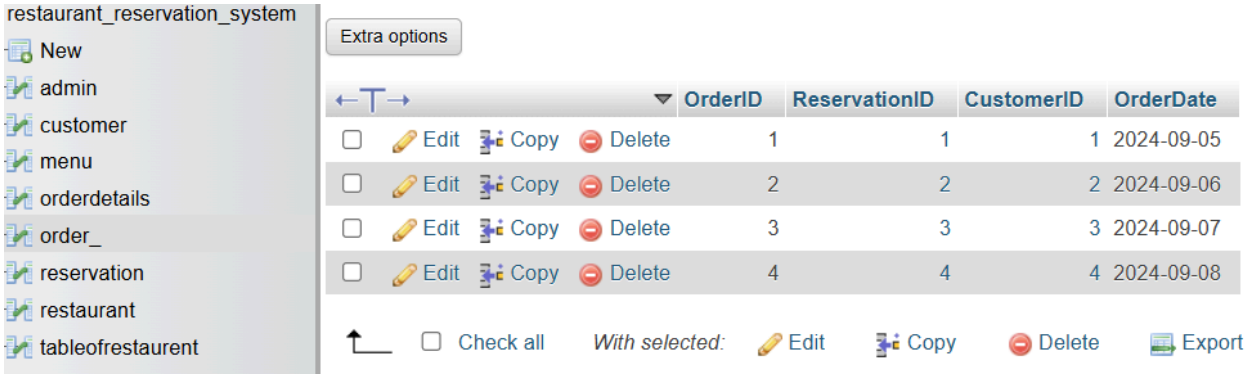


Figure :9

Inserting data into the 'OrderDetails' table

INSERT INTO OrderDetails (OrderID, MenuItemID, Quantity)
VALUES

(1, 1, 2), -- Tasnim Era ordered 2 Chicken Fries
(1, 2, 1), -- Tasnim Era ordered 1 Burger
(2, 3, 1), -- Rakib Hasan ordered 1 Pizza
(2, 5, 2), -- Rakib Hasan ordered 2 Soft Drinks
(3, 4, 1), -- Nadia Islam ordered 1 Pasta
(3, 6, 1), -- Nadia Islam ordered 1 Ice Cream
(4, 1, 3), -- Asif Mahmud ordered 3 Chicken Fries
(4, 3, 2); -- Asif Mahmud ordered 2 Pizzas

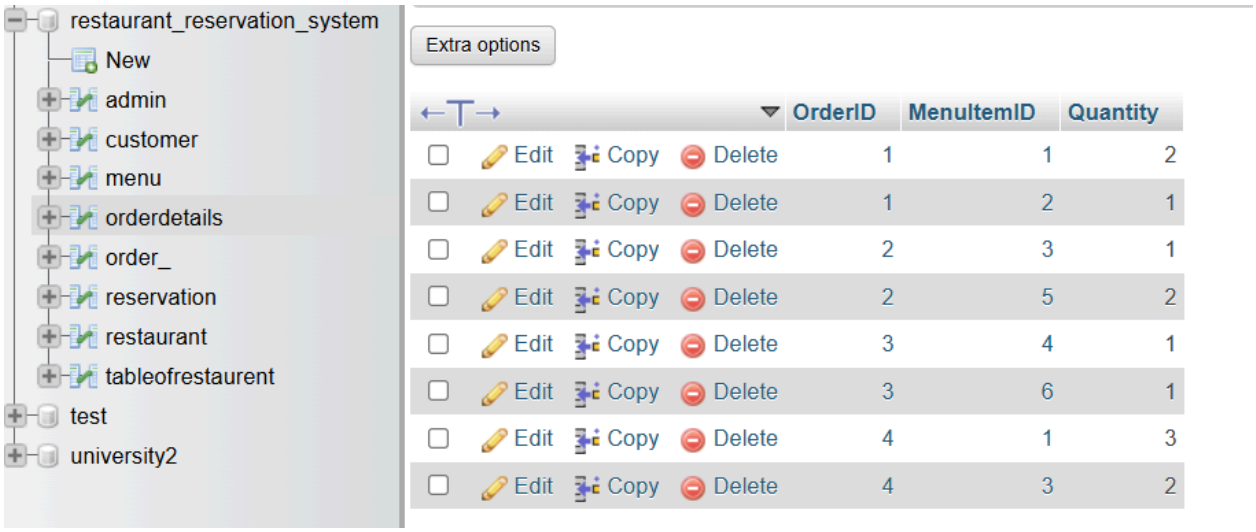
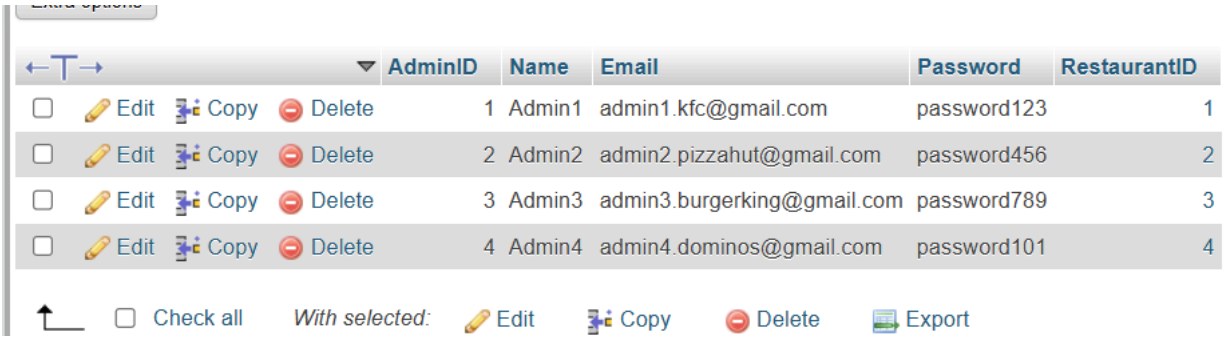


Figure :10

Inserting data into the 'Admin' table

```
INSERT INTO Admin (AdminID, Name, Email, Password, RestaurantID)
VALUES
    (1, 'Admin1', 'admin1.kfc@gmail.com', 'password123', 1), -- Admin for KFC
     (2, 'Admin2', 'admin2.pizzahut@gmail.com', 'password456', 2), -- Admin for Pizza Hut
    (3, 'Admin3', 'admin3.burgerking@gmail.com', 'password789', 3), -- Admin for Burger King
     (4, 'Admin4', 'admin4.dominos@gmail.com', 'password101', 4); -- Admin for Domino's
```



	AdminID	Name	Email	Password	RestaurantID
Edit Copy Delete	1	Admin1	admin1.kfc@gmail.com	password123	1
Edit Copy Delete	2	Admin2	admin2.pizzahut@gmail.com	password456	2
Edit Copy Delete	3	Admin3	admin3.burgerking@gmail.com	password789	3
Edit Copy Delete	4	Admin4	admin4.dominos@gmail.com	password101	4

Check all *With selected:* Edit Copy Delete Export

Figure :11

3. Data Control Language (DCL): DCL includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system.

GRANT Command:

Syntax : GRANT SELECT, INSERT ON Restaurant TO 'user_name';

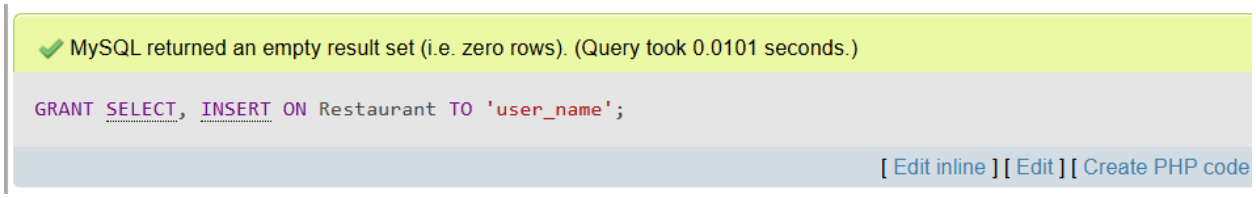


Figure :12

Allows user_name to perform SELECT and INSERT operations on the Restaurant table.

REVOKE Command:

Syntax : REVOKE INSERT ON Restaurant FROM 'user_name';

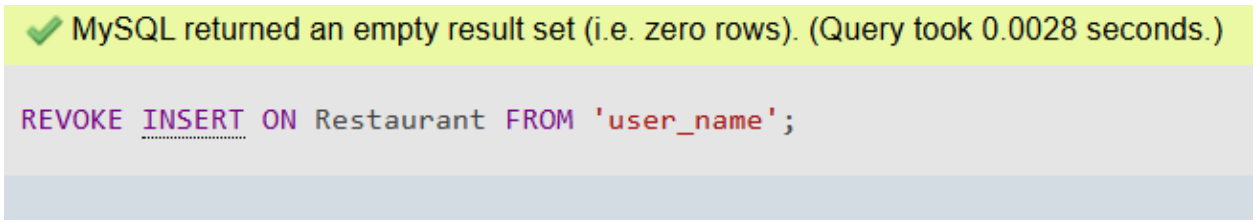


Figure :13

Revokes the INSERT permission from user_name, preventing them from inserting data into the Restaurant table.

4. Transaction Control Language (TCL):

Transactions group a set of tasks into a single execution unit. Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed. If any of the tasks fail, the transaction fails. The key TCL commands are COMMIT, ROLLBACK, SAVEPOINT, and SET TRANSACTION.

COMMIT Command: Permanently saves all changes made in the current transaction.

Syntax: COMMIT;

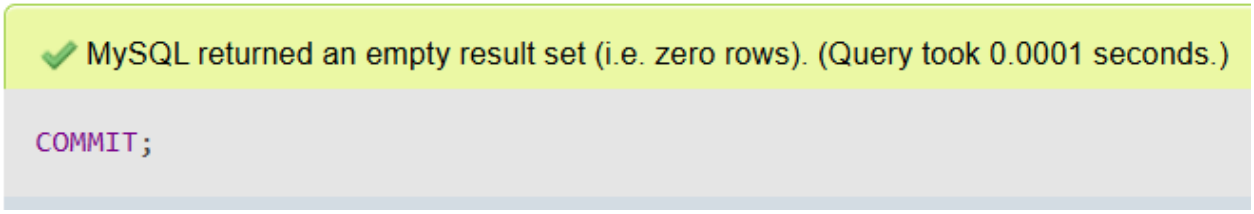


Figure :14

Once issued, all changes in the transaction are saved and cannot be undone.

ROLLBACK Command: Reverts changes made in the current transaction before they are committed.

Syntax: ROLLBACK;

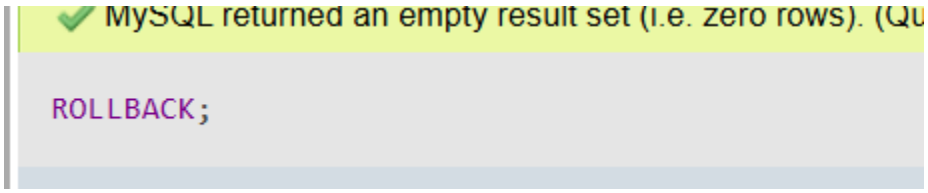


Figure :15

Restores the database to its state before the transaction begins.

SAVEPOINT Command: Sets a savepoint within a transaction to which we can roll back.

Syntax: SAVEPOINT SavepointName;

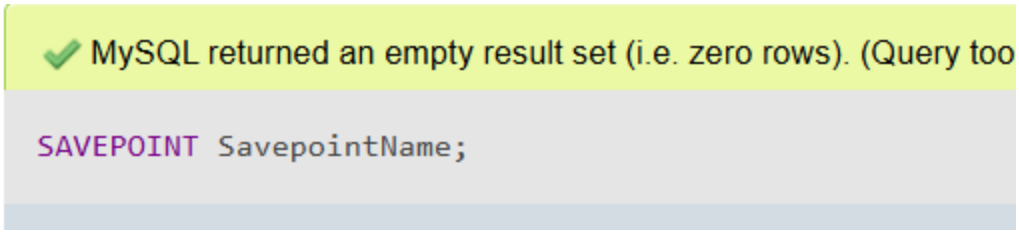


Figure :16

Creates a savepoint in the transaction for granular control.

SET TRANSACTION Command: Defines transaction characteristics, such as the isolation level.

Syntax: SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

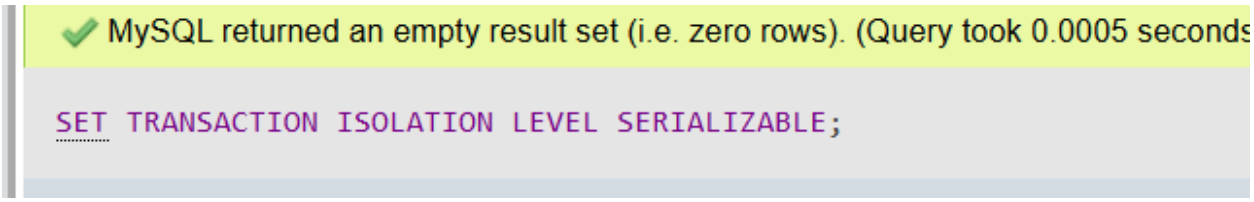


Figure :17

Controls how transactions are isolated from one another.

Queries and Outputs:

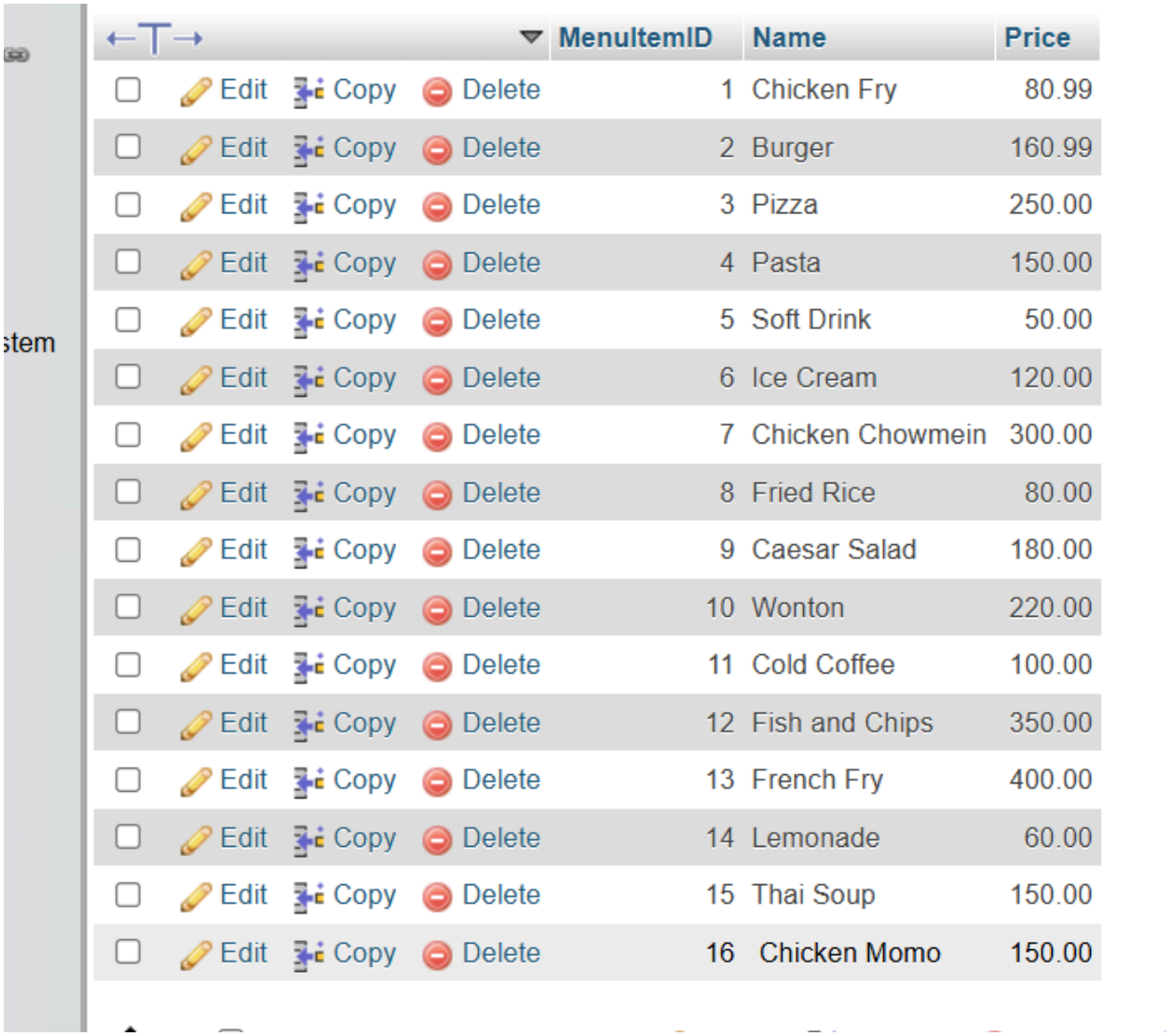
SQL queries:-

1. Save Data (INSERT) : To save or insert new data into a table, an INSERT query is used.

For example, to insert a new menu item with the ID 16, named 'Chicken Momo', priced at 150.00, we can use the following SQL query. The values can be modified accordingly for additional menu items:

Inserting new menu item :

Syntax: INSERT INTO Menu (MenuItemID, Name, Price)
VALUES (16, ' Chicken Momo', 150.00);



	MenuItemID	Name	Price
Edit Copy Delete	1	Chicken Fry	80.99
Edit Copy Delete	2	Burger	160.99
Edit Copy Delete	3	Pizza	250.00
Edit Copy Delete	4	Pasta	150.00
Edit Copy Delete	5	Soft Drink	50.00
Edit Copy Delete	6	Ice Cream	120.00
Edit Copy Delete	7	Chicken Chowmein	300.00
Edit Copy Delete	8	Fried Rice	80.00
Edit Copy Delete	9	Caesar Salad	180.00
Edit Copy Delete	10	Wonton	220.00
Edit Copy Delete	11	Cold Coffee	100.00
Edit Copy Delete	12	Fish and Chips	350.00
Edit Copy Delete	13	French Fry	400.00
Edit Copy Delete	14	Lemonade	60.00
Edit Copy Delete	15	Thai Soup	150.00
Edit Copy Delete	16	Chicken Momo	150.00

Figure :18

Since **MenuItemID** is the primary key and must be unique, each new menu item requires a distinct **MenuItemID**. In this case, the **MenuItemID** for the new menu item is 16, which is not a duplicate.

Inserting two new customers into the Customer table

```
INSERT INTO Customer (CustomerID, Name, Phone, Email)
VALUES (5, 'Jihad', 01712345678, 'jihad@gmail.com'),
 (6, 'Esha', 01812349976, 'esha@gmail.com');
```

	CustomerID	Name	Phone	Email
Edit Copy Delete	1	Tasnim Era	01611744381	jarin@gmail.com
Edit Copy Delete	2	Rakib Hasan	01789834538	rakib@gmail.com
Edit Copy Delete	3	Nadia Islam	01812345678	nadia@gmail.com
Edit Copy Delete	4	Asif Mahmud	01912345678	asif@gmail.com
Edit Copy Delete	5	Jihad	1712345678	jihad@gmail.com
Edit Copy Delete	6	Esha	1812349976	esha@gmail.com

Figure :19

2. Retrieve Data (SELECT):

To access or retrieve data from the database, a SELECT query is used.
For example,

Retrieving all customer details from the Customer table:

Syntax: SELECT * FROM Customer;

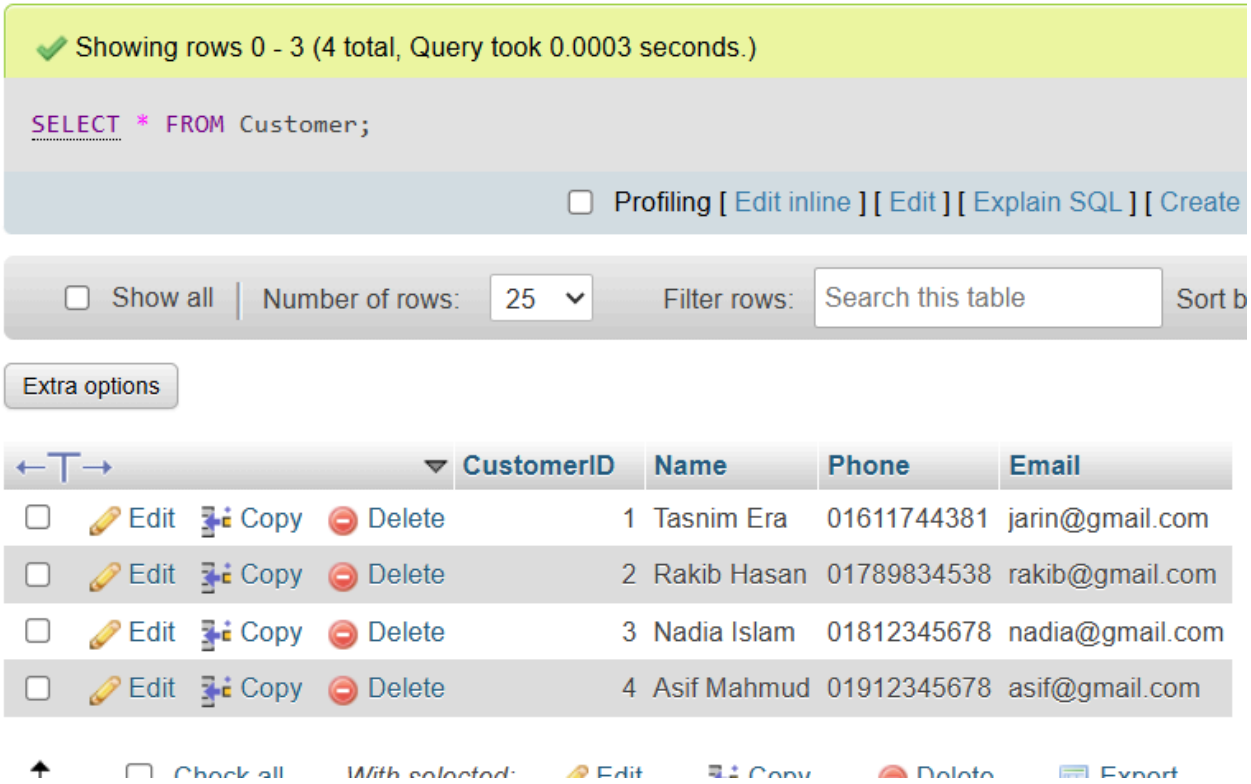


Figure :20

We can also retrieve specific data with conditions.
For example ,

Retrieving OrderID and OrderDate from order_ table:

Syntax: SELECT OrderID,OrderDate
from order_;

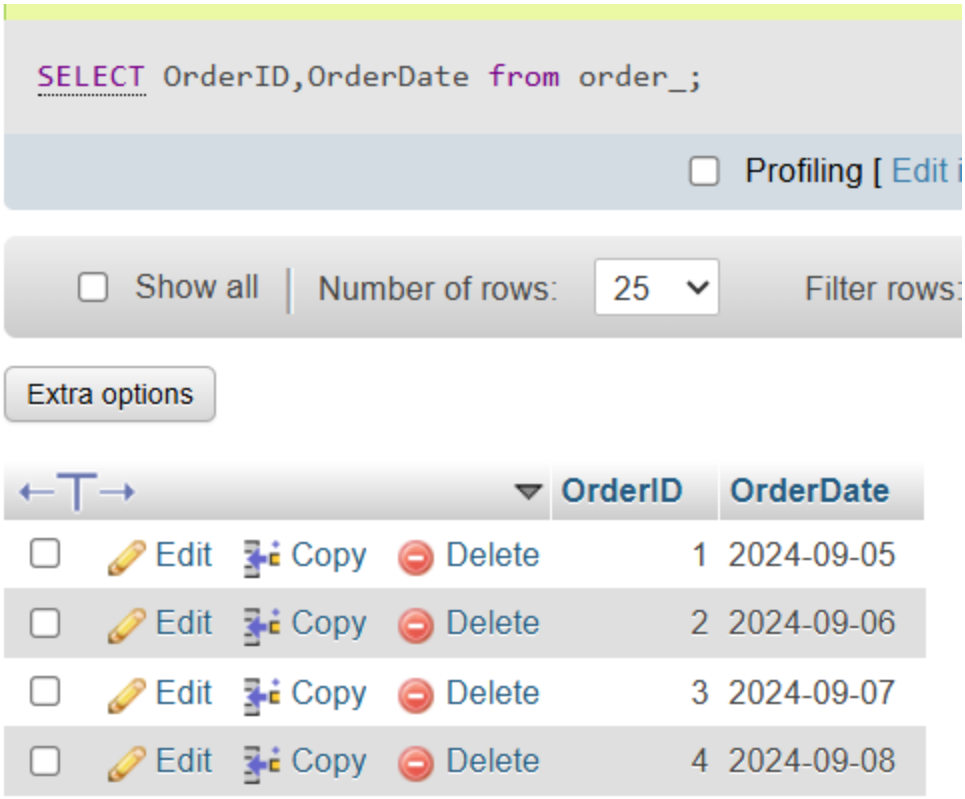


Figure :21

Retrieving a specific customer by ID:

Syntax: SELECT * FROM Customer
WHERE CustomerID = 3;

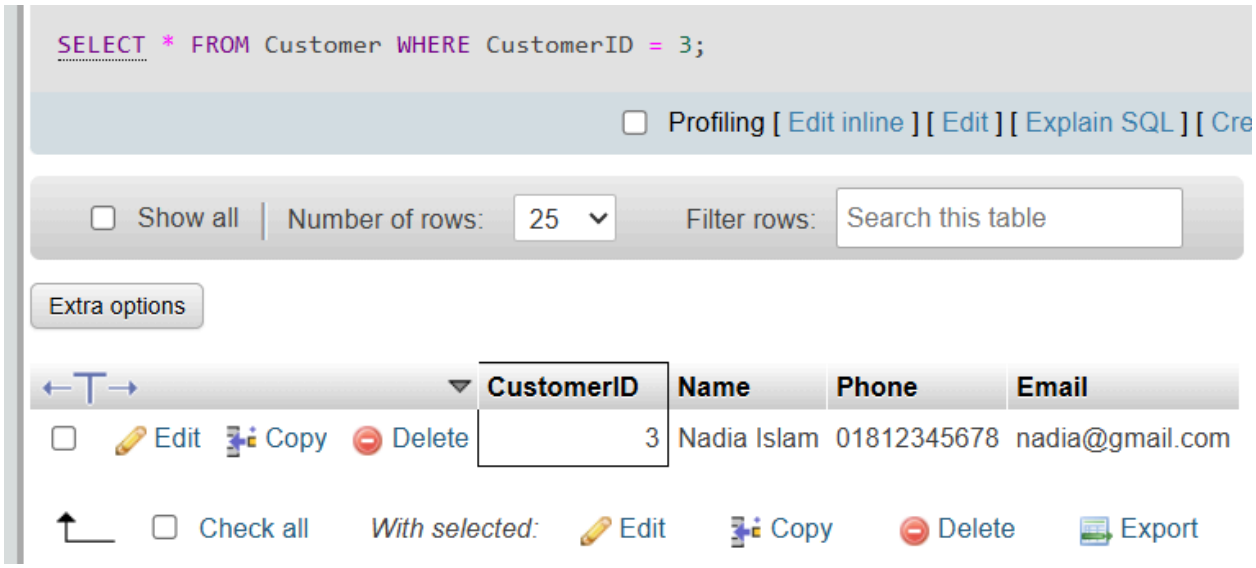


Figure :22

Retrieving the Name as New_name and the CustomerID from the Customer table where the Name starts with the letter "T"

Syntax: SELECT Name AS New_name, CustomerID
FROM Customer
WHERE Name LIKE 'T%';

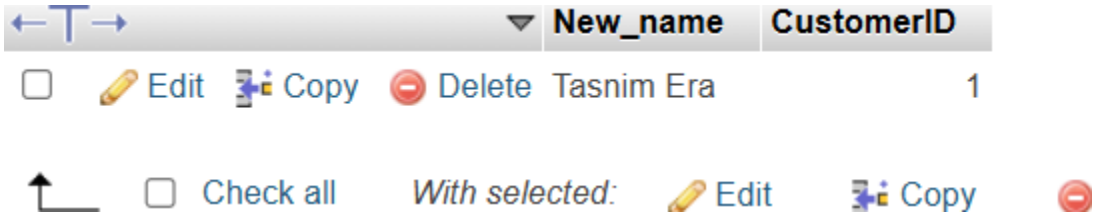


Figure :23

LIKE 'T%': This condition retrieves all rows where the Name starts with "T".
AS New_name: Renames the Name column in the output to New_name.

3. Modify COLUMN -

Add column-

Syntax: ALTER TABLE restaurant
ADD Ranking varchar(30);

Figure :24

Rename the Column name-

Syntax: ALTER TABLE restaurant
CHANGE Ranking rank_NewName char(30);

	RestaurantID	Name	Location	rank_NewName
Edit Copy Delete	1	KFC	Mohammadpur	NULL
Edit Copy Delete	2	Pizza Hut	Dhanmondi	NULL
Edit Copy Delete	3	Burger King	Banani	NULL
Edit Copy Delete	4	Domino's	Gulshan	NULL

Figure :25

Drop column-

Syntax: ALTER TABLE restaurant
DROP COLUMN rank_NewName;

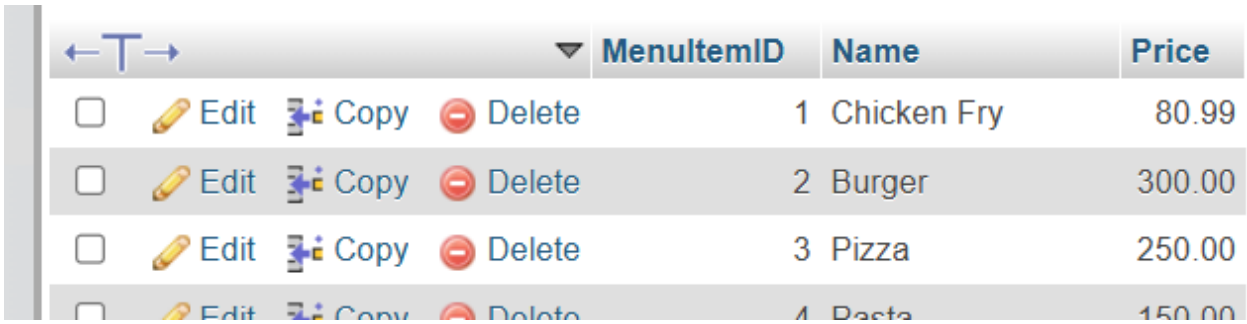
	RestaurantID	Name	Location
Edit Copy Delete	1	KFC	Mohammadpur
Edit Copy Delete	2	Pizza Hut	Dhanmondi
Edit Copy Delete	3	Burger King	Banani
Edit Copy Delete	4	Domino's	Gulshan

Figure :26

4. Update Data (UPDATE)

updating the price of the menu item .

Syntax: UPDATE Menu
SET Price = 300.00
WHERE MenuItemID = 2;



	MenuItemID	Name	Price
Edit Copy Delete	1	Chicken Fry	80.99
Edit Copy Delete	2	Burger	300.00
Edit Copy Delete	3	Pizza	250.00
Edit Copy Delete	4	Pasta	150.00

Figure :27

5. Delete Data:

Deleting two customers information from customer table:

Syntax: DELETE FROM Customer
WHERE CustomerID IN (5,6);

	CustomerID	Name	Phone	Email
Edit Copy Delete	1	Tasnim Era	01611744381	jarin@gmail.com
Edit Copy Delete	2	Rakib Hasan	01789834538	rakib@gmail.com
Edit Copy Delete	3	Nadia Islam	01812345678	nadia@gmail.com
Edit Copy Delete	4	Asif Mahmud	01912345678	asif@gmail.com

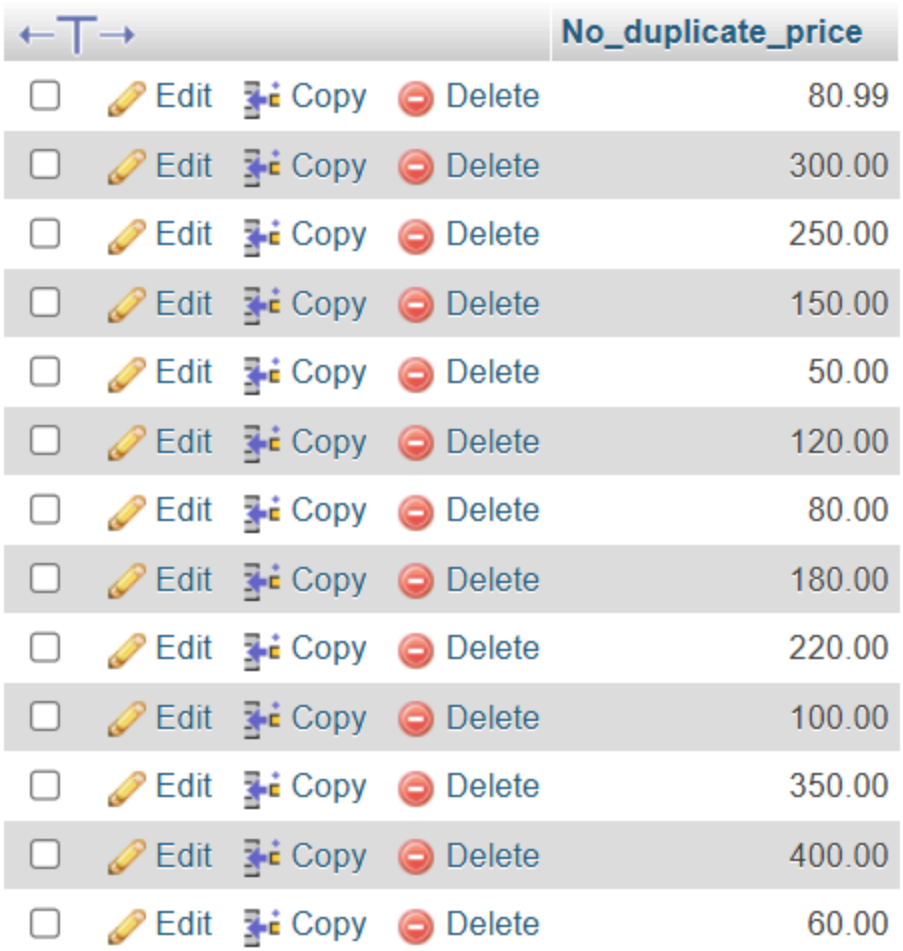
Figure :28

This query deletes the customers with CustomerID 5 and 6 from the **Customer** table.

6. Distinct Keyword:

Retrieving distinct prices from the "Menu" table and displaying the result with the column name No_duplicate_price

Syntax: SELECT DISTINCT Price AS No_duplicate_price
FROM Menu;



←T→				No_duplicate_price
☐	Edit	Copy	Delete	80.99
☐	Edit	Copy	Delete	300.00
☐	Edit	Copy	Delete	250.00
☐	Edit	Copy	Delete	150.00
☐	Edit	Copy	Delete	50.00
☐	Edit	Copy	Delete	120.00
☐	Edit	Copy	Delete	80.00
☐	Edit	Copy	Delete	180.00
☐	Edit	Copy	Delete	220.00
☐	Edit	Copy	Delete	100.00
☐	Edit	Copy	Delete	350.00
☐	Edit	Copy	Delete	400.00
☐	Edit	Copy	Delete	60.00

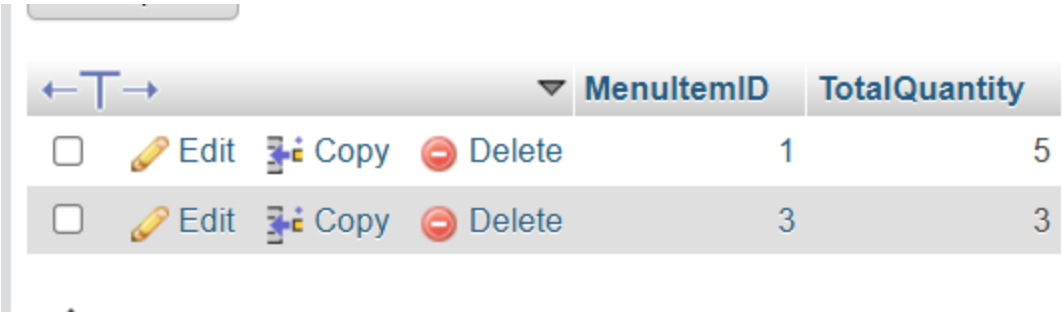
Figure :29

7. Aggregate Functions:

Retrieving data from the "OrderDetails" table.

This query will show the total quantity ordered for each MenuItemID where the total quantity ordered is greater than 2.

Syntax: SELECT MenuItemID, SUM(Quantity) AS TotalQuantity
FROM OrderDetails
GROUP BY MenuItemID
HAVING SUM(Quantity) > 2;



	MenuItemID	TotalQuantity
Edit Copy Delete	1	5
Edit Copy Delete	3	3

Figure :30

Here,SUM(Quantity) calculates the total quantity of each menu item ordered. GROUP BY MenuItemID groups the rows by each unique MenuItemID. HAVING SUM(Quantity) > 2 filters the results to only show menu items where the total quantity ordered is greater than 2.

Finding the average price of menu items for each unique price value that occurs more than once:

Syntax: SELECT Price AS Menu_Price, COUNT(*) AS Count_of_Price, AVG(Price) AS Average_Price
FROM Menu
GROUP BY Price
HAVING COUNT(*) > 1;

Menu_Price	Count_of_Price	Average_Price
150.00	3	150.000000
300.00	2	300.000000

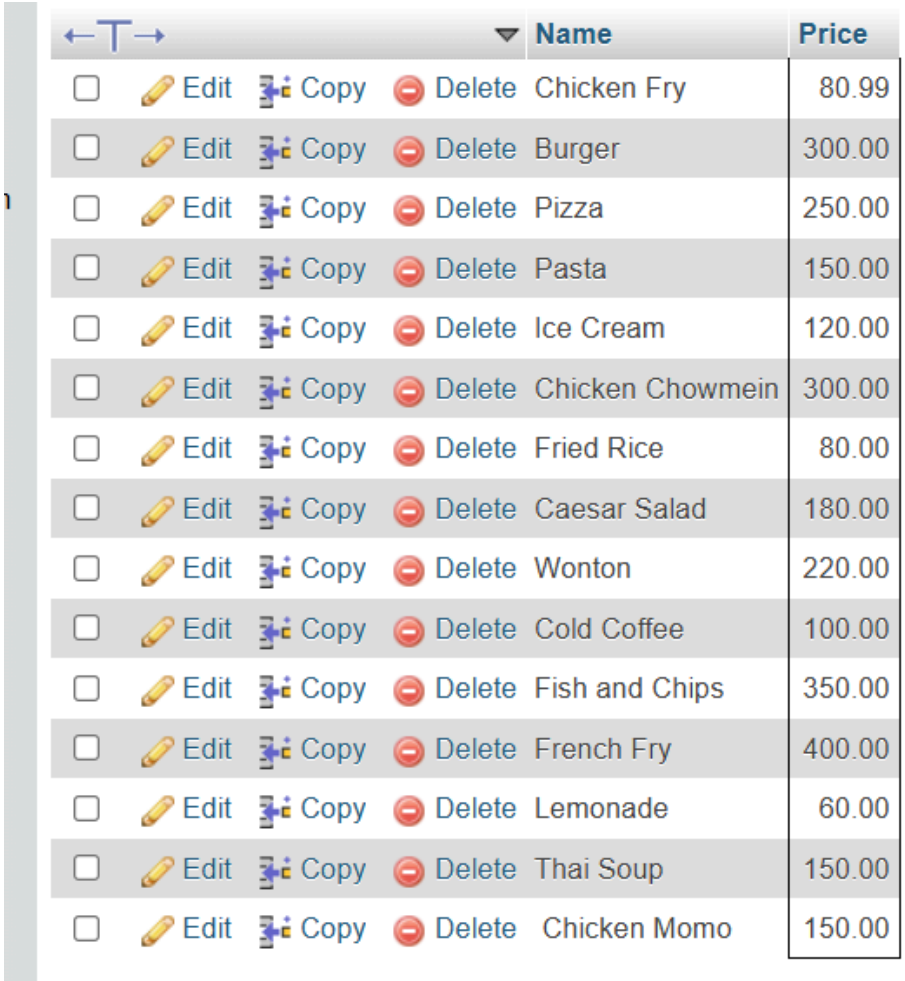
Figure :31

In this query, COUNT(*) counts occurrences of each price, AVG(Price) calculates the average price (same as the price itself per group), GROUP BY Price organizes results by each distinct price, and HAVING COUNT(*) > 1 filter to show prices that appear more than once in the "Menu" table.

8. Subqueries /Nested Queries:

To Find menu items where the price is greater than the price of at least one other menu item:

Syntax: SELECT Name, Price
FROM Menu
WHERE Price > ANY (
    SELECT Price
    FROM Menu
    WHERE Price < (SELECT MAX(Price) FROM Menu)
);



	Name	Price
Edit Copy Delete	Chicken Fry	80.99
Edit Copy Delete	Burger	300.00
Edit Copy Delete	Pizza	250.00
Edit Copy Delete	Pasta	150.00
Edit Copy Delete	Ice Cream	120.00
Edit Copy Delete	Chicken Chowmein	300.00
Edit Copy Delete	Fried Rice	80.00
Edit Copy Delete	Caesar Salad	180.00
Edit Copy Delete	Wonton	220.00
Edit Copy Delete	Cold Coffee	100.00
Edit Copy Delete	Fish and Chips	350.00
Edit Copy Delete	French Fry	400.00
Edit Copy Delete	Lemonade	60.00
Edit Copy Delete	Thai Soup	150.00
Edit Copy Delete	Chicken Momo	150.00

Figure :32

In this query, the subquery [SELECT Price FROM Menu WHERE Price < (SELECT MAX(Price) FROM Menu)] retrieves all prices less than the maximum price, while the outer query selects menu items with prices greater than any of these prices.

Finding menu items where the price is greater than all other menu items:

Syntax: SELECT Name, Price
FROM Menu
WHERE Price > ALL (
  SELECT Price
  FROM Menu
  WHERE Price < (SELECT MAX(Price) FROM Menu)
);

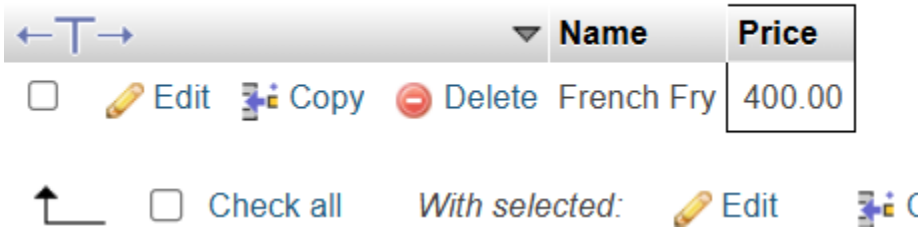


Figure :33

In this query, the subquery retrieves all prices less than the maximum price, and the outer query selects menu items with prices greater than all these prices.

9. Correlated Subqueries and Exists:

A correlated subquery references columns from the outer query.

Find menu items that are priced higher than any menu item in the same table:

Syntax: SELECT Name, Price
FROM Menu AS m1
WHERE Price > (
    SELECT MAX(Price)
    FROM Menu AS m2
    WHERE m1.MenuItemID <> m2.MenuItemID
);

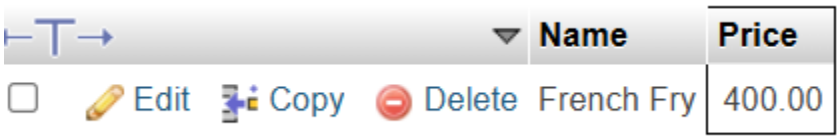


Figure :34

In this query, the subquery calculates the maximum price of items other than the current item, and the outer query selects menu items with prices greater than this maximum.

Find menu items where there exists at least one menu item with a higher price:

Syntax: SELECT Name, Price
FROM Menu AS m1
WHERE EXISTS (
    SELECT 1
    FROM Menu AS m2
    WHERE m2.Price > m1.Price
);

	Name	Price
Edit Copy Delete	Chicken Fry	80.99
Edit Copy Delete	Burger	300.00
Edit Copy Delete	Pizza	250.00
Edit Copy Delete	Pasta	150.00
Edit Copy Delete	Soft Drink	50.00
Edit Copy Delete	Ice Cream	120.00
Edit Copy Delete	Chicken Chowmein	300.00
Edit Copy Delete	Fried Rice	80.00
Edit Copy Delete	Caesar Salad	180.00
Edit Copy Delete	Wonton	220.00
Edit Copy Delete	Cold Coffee	100.00
Edit Copy Delete	Fish and Chips	350.00
Edit Copy Delete	Lemonade	60.00
Edit Copy Delete	Thai Soup	150.00
Edit Copy Delete	Chicken Momo	150.00

Console

Figure :35

10. Join operation

INNER JOIN query to retrieve data from two tables in your Restaurant Reservation System

Syntax: SELECT Customer.Name, Customer.Phone, Reservation.ReservationDate, Reservation.ReservationTime
FROM Customer
INNER JOIN Reservation ON Customer.CustomerID = Reservation.CustomerID;

Extra options

Name	Phone	ReservationDate	ReservationTime
Tasnim Era	01611744381	2024-09-05	19:00:00
Rakib Hasan	01789834538	2024-09-06	18:00:00
Nadia Islam	01812345678	2024-09-07	19:30:00
Asif Mahmud	01912345678	2024-09-08	20:00:00

Figure :36

This query retrieves data from the **Customer** and **Reservation** tables using an **INNER JOIN**. It returns the **Customer Name**, **Phone**, **Reservation Date**, and **Reservation Time** for all customers who have made reservations. The **INNER JOIN** ensures that only rows where the CustomerID in the **Customer** table matches the CustomerID in the **Reservation** table are included in the result. If a customer does not have a corresponding reservation, their data will not appear in the result.

LEFT JOIN Query:

Syntax: SELECT Customer.Name, Customer.Phone, Reservation.ReservationDate, Reservation.ReservationTime
FROM Customer
LEFT JOIN Reservation ON Customer.CustomerID = Reservation.CustomerID;

Name	Phone	ReservationDate	ReservationTime
Tasnim Era	01611744381	2024-09-05	19:00:00
Rakib Hasan	01789834538	2024-09-06	18:00:00
Nadia Islam	01812345678	2024-09-07	19:30:00
Asif Mahmud	01912345678	2024-09-08	20:00:00

Figure :37

This query retrieves all customers, including those who do not have any reservations. The **LEFT JOIN** ensures that all rows from the **Customer** table are included, even if there is no matching CustomerID in the **Reservation** table. For customers without reservations, the ReservationDate and ReservationTime will appear as NULL.

RIGHT JOIN Query:

Syntax: SELECT Customer.Name, Customer.Phone, Reservation.ReservationDate, Reservation.ReservationTime
FROM Customer
RIGHT JOIN Reservation ON Customer.CustomerID = Reservation.CustomerID;

Name	Phone	ReservationDate	ReservationTime
Tasnim Era	01611744381	2024-09-05	19:00:00
Rakib Hasan	01789834538	2024-09-06	18:00:00
Nadia Islam	01812345678	2024-09-07	19:30:00
Asif Mahmud	01912345678	2024-09-08	20:00:00

Figure :38

This query retrieves all reservations, including those where no matching customer exists. The **RIGHT JOIN** ensures that all rows from the **Reservation** table are included, even if there is no matching CustomerID in the **Customer** table. For reservations without corresponding customers, the Name and Phone fields will appear as NULL.

Full join is not supported in mysql.

Normalization :

The Restaurant Reservation System database follows three key stages of normalization: 1NF, 2NF, and 3NF.

In **First Normal Form (1NF)**, all tables have atomic values, meaning each cell contains a single piece of data with no repeating groups. For example, the Menu table holds atomic values for MenuItemID, Name, and Price.

In **Second Normal Form (2NF)**, all non-key attributes depend on the entire primary key. For instance, in the Customer table, attributes like Name, Phone, and Email depend fully on CustomerID, the primary key.

In **Third Normal Form (3NF)**, non-key attributes depend only on the primary key, not on other non-key attributes. For example, in the Admin table, attributes like Name, Email, and Password depend only on AdminID.

The database is normalized up to 3NF, eliminating redundancy, ensuring data integrity, and optimizing performance for data retrieval and updates.

Modern tools used:

For the development of the **Restaurant Reservation System**, modern tools were employed to ensure efficient database management and system design.

The database was created and managed using **XAMPP**, a popular open-source platform that integrates **MySQL** for handling database queries and operations, all running on **localhost** to simulate a real-world environment for testing and development. Additionally, the **ER diagram** (Entity-Relationship Diagram), which visually represents the database schema and relationships between entities, was designed using **draw.io**, a widely-used online diagramming tool. These tools provided a comprehensive and user-friendly environment for the successful design, implementation, and visualization of the database.

Conclusion:

The **Restaurant Reservation System** database effectively manages key aspects of a restaurant's operations, such as customer information, table reservations, menus, and orders. By organizing the data into well-structured tables with appropriate relationships, the system ensures efficient handling of reservations, customer orders, and administrative functions. The use of **foreign keys** and constraints enforces data integrity, while **SQL queries** provide flexibility for retrieving and managing data. Overall, this database is designed to streamline the reservation process, enhance customer service, and improve restaurant management.